Data-Dependent Selection of Low-Precision MXFP / MXINT / BFP Numerical Formats

Generic Algorithm 1: *Block-Importance Selector*
Generic Algorithm 2: *Sampled-Contribution Selectors*
Generic Algorithm 3: *Histogram Selector (GHS-2)*
Generic Algorithm 4: *Calibration-Column Sampling Selector*

Choosing a 4-bit format from part of the work, compared with brute force

Course	CS611
Project	Data-Dependent Selection of Low-Precision MXFP/MXINT/BFP Numerical Formats
Assignment	Assignment 1 (group assignment)
Group / data	Group 7 | Dataset C | $W \in \mathbb{R}^{100\times100}$, $X_{\text{cal}} \in \mathbb{R}^{100\times256}$ | 4-bit elements
Date	October 25, 2026

Group members

#	Name	Roll number
1	Naitikkumar H Patel	261110039
2	S Aditya	261110057
3	Jaimin Gajera	261110028
4	Krish Kalgude	261110030
5	Krunal Rakholiya	261110032

Abstract

We compare four low-cost selectors for choosing among five 4-bit formats using calibration-output error. Algorithms 1–4 use important weight blocks, sampled products, histogram-guided refinement, and one maximum-energy input column plus 31 random columns, respectively. On Dataset C, all select MXFP E2M1, matching the exhaustive reference. Algorithms 1 and 4 use 39.0% and 13.9% of baseline operations, while Algorithm 3 matches the exhaustive-search error (12.031%) using 52.7% of its roundings.

Index

1 What the data look like **3**

2 The five candidate formats: definitions and research references **3**

3 Generic Algorithm 1: the Block-Importance Selector (wt_x) **5**

3.1 How it works . . . 5

3.2 Result against brute force . . . 6

4 Generic Algorithm 2: the Sampled-Contribution Selectors **7**

4.1 Shared setup . . . 7

4.2 Version 1: column-wise K . . . 7

4.3 Version 2: top entries of W and of X_{cal} . . . 7

4.4 Version 3: top columns of W and top rows of X_{cal} . . . 8

4.5 Result on Dataset C . . . 8

5 Generic Algorithm 3: the Histogram Selector (GHS-2) **9**

5.1 How it works . . . 9

5.2 Result against brute force . . . 10

5.3 Other inputs and limits . . . 10

6 Generic Algorithm 4: the Calibration-Column Sampling Selector **11**

6.1 How it works . . . 11

6.2 Result against brute force . . . 12

References **12**

List of Tables

1 Profile of W (Dataset C). . . . 3

2 Observed percentiles of $|W|$ against Laplace and Gaussian laws . . . 3

3 Candidate formats and their sources . . . 3

4 Dataset C: selector against the full baseline and exhaustive search . . . 6

5 Dataset C: work of the selector against the two references . . . 6

6 Dataset C: sampled selectors against the full baseline and exhaustive search . . . 8

7 Dataset C: Algorithm 3 stage by stage against the full baseline and exhaustive search 10

8 Dataset C: Algorithm 4 against the full baseline and exhaustive search . . . 12

1 What the data look like

Table 1 profiles W. Two things stand out: the weights are strongly peaked at zero (median $|w| = 0.49$ against 0.67 for a Gaussian of the same standard deviation) and they have a heavy tail (max $|w| = 5.7$, i.e. 5.7σ). Table 2 compares the observed quantiles to a Laplace distribution with unit variance (scale $b = 1/\sqrt{2}$): they agree within 2%, while a Gaussian would be off by 25–30% at the median and the 99th percentile. We therefore treat Dataset C as *Laplace-like* ("consistent with", not proven); this observation drives most of the differences between the formats below.

Statistic	Value	Statistic	Value
Shape of W / X_{cal}	100×100 / 100×256	P90 $\lvert w\rvert$	1.631
Mean / Std	0.0047 / 1.0000	P95 $\lvert w\rvert$	2.081
Negative weights	4 948 (49.48%)	P99 $\lvert w\rvert$	3.289
Exact zeros	0%	Max $\lvert w\rvert$	5.700
$\lvert w\rvert < 0.1\sigma$	13.22%	Median $\lvert w\rvert$	0.488

Table 1: Profile of W (Dataset C).

Quantity	Observed (Dataset C)	Laplace, $b = 1/\sqrt{2}$	Gaussian $N(0,1)$
Median $\lvert w\rvert$ (P50)	0.488	0.490	0.674
P90 $\lvert w\rvert$	1.631	1.628	1.645
P95 $\lvert w\rvert$	2.081	2.118	1.960
P99 $\lvert w\rvert$	3.289	3.256	2.576
$\Pr(\lvert w\rvert < 0.1)$	0.1322	0.1319	0.0797

Table 2: Observed percentiles of $|W|$ versus theoretical Laplace and Gaussian laws of unit variance.

2 The five candidate formats: definitions and research references

Our implementation keeps two kinds of information apart: what a standard or a paper fixes, and what the project had to choose itself. Only MXFP E2M1 is an OCP-standard element format. E1M2 and BFP4 come from research papers, MXINT4 is the course convention and MXUINT4 is our own encoding. Every ranking in this report is therefore *conditional* on the conventions stated here.

Common skeleton. Weights are grouped into blocks of up to 32 consecutive elements along a row. Each block stores 4-bit element codes and one 8-bit shared exponent. A 100×100 matrix has 400 blocks (the last block of each row holds only 4 weights), so every format costs $4 + 3200/10000 = 4.32$ bits per weight. Rounding is to nearest with ties to even, values beyond the largest level saturate, the data are finite so no format produces NaN or Inf, and a weight is reconstructed as $\hat{w} = (\text{level}) \times 2^e$. We wrote the quantisers ourselves; no library low-precision routine is used.

Format	Levels (shared exponent e)	Baseline exponent	Source and status
MXFP E2M1	$\pm\{0, .5, 1, 1.5, 2, 3, 4, 6\} \cdot 2^e$	$\lfloor \log_2 a_{\max} \rfloor - 2$	OCP MX v1.0 (standard)
MXFP E1M2	$\pm\{0, .5, 1, \ldots, 3.5\} \cdot 2^e$	$\lfloor \log_2 a_{\max} \rfloor - 1$	Wang et al., ICML 2025; block scaling is a project convention
MXINT4	$\{-8, \ldots, 7\} \cdot 2^{e-2}$	$\lfloor \log_2 a_{\max} \rfloor$	course / project convention
MXUINT4	$(q_u - 8) \cdot 2^e$, $q_u \in [0, 15]$	$\lceil \log_2 \max(\frac{\lfloor \min \rfloor}{8}, \frac{\max}{7}) \rceil$	custom, project-defined
BFP4	$\{-8, \ldots, 7\} \cdot 2^E$	$\lceil \log_2 a_{\max} \rceil - 3$	Köster et al., NeurIPS 2017; shared per block

Table 3: Candidate formats. $a_{\max}$ is the largest magnitude in the block.

MXFP E2M1: OCP standard. Defined in the OCP Microscaling Formats (MX) Specification v1.0, Sections 5.2 and 5.3.4 [1, 2]. The element codebook is $\pm\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$, the canonical block has 32 elements and the shared scale is an 8-bit E8M0 exponent. The baseline exponent $\lfloor \log_2 a_{\max} \rfloor - 2$ places the block maximum in $[4, 8) \cdot 2^e$, at the top of the grid.

MXFP E1M2: paper codebook. Wang et al., *Optimizing Large Language Model Training Using FP4 Quantization*, ICML 2025 (PMLR 267), Table 4 [3], lists the codebook $\pm\{0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5\}$. It is not a concrete format of OCP MX v1.0; the block scaling used here, with baseline exponent $\lfloor \log_2 a_{\max} \rfloor - 1$, is a separately declared microscaling convention.

BFP4: Flexpoint, adopted from the course reference. Köster et al., NeurIPS 2017, arXiv:1711.02213 [4]. Flexpoint flex$N$+M stores each element as an N-bit two's-complement integer mantissa and shares one M-bit exponent across the group, so the value is $m \cdot \kappa$ with $\kappa = 2^E$. Here $N = 4$ (codes $-8, \ldots, 7$) and $M = 8$ (an E8M0-style exponent, 8 bits of metadata per group). The shared exponent follows the paper's Autoflex rule $E = \lceil \log_2 \chi \rceil - N + 1$ (Algorithm 2), with χ the group maximum: a static weight block is a single observation with standard deviation 0 and $\alpha = 1$, because static weights do not drift (the setting `BFP_EXP_POLICY="autoflex_init"` runs the paper's Algorithm 1 literally). The paper does not fix the rounding and names stochastic rounding as an alternative, so we use the assignment's round-to-nearest-even with saturation. The paper shares one exponent per tensor; we use one per 32-element row-wise block, as for the MX candidates and as the course describes BFP (`BFP_SHARING="tensor"` gives the literal tensor-wide variant).

MXINT4: course convention. The assignment asks for MXINT4, but the concrete integer format in the OCP MX v1.0 format table is MXINT8. We therefore treat the 4-bit signed integer range $[-8, 7]$, the 2^{-2} element precision scale and the per-block power-of-two scale as the course/project convention. They are recorded in our format configuration (`FORMAT_CONFIGS`) and should be replaced only if the lecture specifies another rule.

MXUINT4: custom midpoint-coded unsigned storage. 49.48 % of W is negative, so a plain unsigned code cannot store it. We store an unsigned code $q_u \in [0, 15]$, decode it as the signed code $q_s = q_u - 8 \in [-8, 7]$ and reconstruct $\hat{w} = q_s \cdot 2^e$. The zero point is the fixed constant 8, so no per-block FP32 offset is stored; only the 8-bit shared exponent per 32-weight block is counted, and decoding costs one integer subtraction per element. This is a project-defined encoding, not a standardised OCP MXUINT4 format.

Important interpretation. Numerical rankings are conditional on the conventions above. The Flexpoint mantissa is a 4-bit two's-complement integer times a shared power of two, so its grid $\{-8, \ldots, 7\} \cdot 2^E$ coincides with the MXINT4 grid $\{-8, \ldots, 7\} \cdot 2^{e-2}$ and the MXUINT4 grid $\{-8, \ldots, 7\} \cdot 2^e$ as soon as the exponent is searched (and the searched windows overlap). The three formats then differ only through the baseline exponent rule and the window around it. Dataset C shows this in Table 4: after the search MXINT4, MXUINT4 and BFP4 all give 13.452 %, while at baseline exponents they give 15.808 %, 17.126 % and 15.808 %. E1M2 offers the same integer grid without the -8 code (15.817 % at baseline, 14.004 % after the search), so only E2M1, with its non-uniform grid, is genuinely different.

3 Generic Algorithm 1: the Block-Importance Selector (wt_x)

Brute force has to quantise all 10 000 weights in all five formats (50 000 values) before it can compare them (the full baseline), and the exhaustive search repeats this for many exponent choices. Generic Algorithm 1 starts from a simple observation: the output error is not spread evenly over W. Some blocks of 32 weights hold large weights on strongly excited inputs, and rounding errors there are expected to matter far more than elsewhere. The algorithm therefore (i) ranks the blocks by importance, (ii) lets the five formats compete on the most important blocks only, and (iii) quantises the remaining blocks for the winning format alone. A weight's importance combines the weight (wt) with the energy of its input (x), hence the name **Block-Importance Selector**, written "wt_x" in the notebook.

3.1 How it works

Step 1: importance of every weight. Row j of X_{cal} holds the 256 calibration values of input feature j, and its energy a_j tells how strongly that input excites the layer. Each weight gets

$$a_j = \sum_{k=1}^{256} X_{jk}^2, \qquad I_{ij} = W_{ij}^2 \, a_j.$$

An error ε in W_{ij} adds $\varepsilon \, X_{\text{cal}}[j, :]$ to row i of the output, an energy of $\varepsilon^2 a_j$, so I_{ij} is large when the weight is large *and* its input is active. It is a proxy for the damage rounding can do, not a bound. On Dataset C the input energies are nearly flat (between 0.81 and 1.21 times their mean), so here the importance is driven mostly by the weights themselves.

Step 2: group the weights into blocks. We use the quantisers' own row-wise blocks of 32 weights (400 blocks; the last block of a row holds 4). For a block b let

$$B_b = \sum_{(i,j) \in b} I_{ij} \text{ (total importance)}, \qquad \pi_b = \max_{(i,j) \in b} I_{ij} \text{ (priority)}.$$

Blocks are ranked by the *maximum*, not the sum, because the largest weight fixes the block exponent and with it the rounding step of all 32 members.

Step 3: choose whole blocks. Sort the blocks by π_b (equal priority: lower index first) and add them, whole, until

$$\sum_{b \in S} B_b \;\geq\; \theta \sum_{b} B_b, \qquad \theta = 0.5.$$

This is half of the *importance*, not half of the weights, and it does not promise that half of the output error is captured. On Dataset C it takes 122 of 400 blocks (3 764 weights, 37.6 %) and covers 50.3 % of the importance, slightly over half because blocks are taken whole.

Step 4: score each format on the selected blocks only. Every format quantises the blocks in S at their *baseline* exponent, computed from the *complete* original block, so a selected block is quantised exactly as in a full pass. The errors go into a matrix Δ_f that is zero elsewhere, and the score is the output RMSE they cause:

$$s_f = \sqrt{\text{mean}\left((\Delta_f X_{\text{cal}})^2\right)}.$$

It includes the interactions among the selected blocks' errors and omits the errors of all other blocks, so it ranks the formats but is not the true error.

Step 5: decide and finish. The smallest s_f wins. Scores are tied if $|s - s'| \leq 10^{-15} + 10^{-6}\max(|s|, |s'|)$; tied finalists are quantised completely and compared by the full output RMSE, and that work is counted. On Dataset C there is no tie. Only now are the 278 remaining blocks (6 236 weights) quantised, for the winning format alone.

3.2 Result against brute force

The *full baseline* quantises all 400 blocks of the five formats at baseline exponents and scores the full output. The *exhaustive search* also searches every block exponent exactly in a window around the baseline (Part 1) and defines the best format. Together they are the brute-force references. Errors are $100\,\|(W - W_{\mathrm{q}})X_{\mathrm{cal}}\|_F / \|WX_{\mathrm{cal}}\|_F$; tied formats share a rank.

	Selector		Full baseline		Exhaustive search	
Format	score	rank	error (%)	rank	error (%)	rank
E2M1	**0.0933**	1	**12.552**	1	**12.031**	1
MXINT4	0.1214	2	15.808	2	13.452	2
BFP4	0.1214	2	15.808	2	13.452	2
E1M2	0.1215	4	15.817	4	14.004	5
MXUINT4	0.1356	5	17.126	5	13.452	2

Table 4: Dataset C. The selector's score is the output RMSE of the selected blocks (not a percentage); the full-baseline and exhaustive-search columns are full output errors.

Dataset C	Full baseline	Exhaustive search	Selector
Weights scored	10 000	10 000	3 764 (37.6 %)
Values quantised	50 000	350 400	25 056 (50.1 %)
Operations (FLOPs)	26 078 012	not counted	10 173 411 (39.0 %)

Table 5: Dataset C. Selector percentages are of the full baseline (values: 50 000; FLOPs: 26 078 012).

Same winner, with a clear margin. E2M1 is rank 1 for the selector, the full baseline and the exhaustive search. Its lead over the runner-up is 30.2 % in the selector's score, 25.9 % at baseline exponents and 11.8 % after the search, so the choice is not a near-tie.

Less work. The selector scores 37.6 % of the weights and quantises 25 056 values, 50.1 % of the baseline's 50 000 and 7.2 % of the search's 350 400. Its operation count is 39.0 % of the baseline's (additions, subtractions, multiplications and divisions in the notebook's model, not timings).

A format, not exponents. The delivered matrix is the baseline E2M1 matrix with error 12.552 %; the exponent search lowers it to 12.031 %, which the selector does not attempt. Its ranking equals the baseline's ($\tau_b = 1.00$) but not the exhaustive one ($\tau_b = 0.63$): MXUINT4 is last at baseline exponents but ties MXINT4 and BFP4 after the search.

Advantage: During the stress testing , this algorithm's worst performed in a dataset where it selected a data format which was around 37% worse RMSE than the best data format.This is much lesser other algorithms tested(which ranged from 76% to 502%). Hence , even when it picks a bad data format , it is never 'too bad'.

4 Generic Algorithm 2: the Sampled-Contribution Selectors

Algorithm 2 attacks the same cost from the other side. The layer output is a sum of contributions, $(WX_{\text{cal}})_{rj} = \sum_i W_{ri}X_{ij}$, one per input feature i. Instead of quantising every weight and evaluating the full product for each format, a selector *samples* the contributions that carry most of the output energy, quantises only the weights they involve, and ranks the formats by the error of that partial product.

4.1 Shared setup

Feature i is column i of W (100×100) and row i of X_{cal} (100×256). The five formats each use 4 bits per weight and one 8-bit exponent per row-wise block of 32 weights. A cheap selector uses the baseline exponent of each *complete* original block and sends only the selected entries to the quantiser; the exhaustive search tries a window of nearby exponents and keeps the choice with the smaller full output error $100\,\|(W - W_{\text{q}})X_{\text{cal}}\|_F / \|WX_{\text{cal}}\|_F$.

4.2 Version 1: column-wise K

Every weight column is scored by the sum of absolute weights, $s_W(i) = \sum_r |W_{ri}|$, and the top 27.5 % are kept: $\lceil 27.5 \rceil = 28$ of 100 columns, so 28.0 % of the weights (2 800) are quantised. A format is scored on the error of the reduced product $W_{:,S}\,X_{\text{cal}}[S,:]$, where S is the set of selected columns.

Advantage. The product uses 28 columns of W and the 28 matching rows of X_{cal} instead of all 100, and only the selected weights are quantised, so both the quantisation and the matrix product shrink.

Disadvantage. The score never looks at X_{cal}: a column with a moderate absolute-weight sum is dropped even when its calibration row is large, and that product still carries a large share of the output energy.

4.3 Version 2: top entries of W and of X_{cal}

Each matrix is scanned on its own. A boolean mask M_W marks the largest 20 % of the entries of $|W|$ and a mask M_X the largest 20 % of the entries of $|X_{\text{cal}}|$. A term $W_{ri}X_{ij}$ is kept when M_W is true at (r, i), or M_X is true at (i, j), or both (counted once); on Dataset C that is 36.0 % of all terms. Each format is ranked by the error on those terms only.

Advantage. The score touches only the selected terms, so there is no full $100 \times 100 \times 256$ product per candidate format.

Disadvantage. The X_{cal} mask is entry-wise, but a weight column is shared by every calibration column: one selected entry in row i of X_{cal} activates every product $W_{ri}X_{ij}$ along that row, so every weight of column i must be quantised, even those not among the largest of W. On Dataset C this reaches all 10 000 weights.

4.4 Version 3: top columns of W and top rows of X_{cal}

Absolute values are taken before summing, so positive and negative entries cannot cancel: for each feature i,

$$s_W(i) = \sum_r |W_{ri}|, \qquad s_X(i) = \sum_j |X_{ij}|.$$

The top 20 % of the weight columns and of the calibration rows give two lists of 20 indices, and the features used for scoring are their *union* (a feature is kept if its weight column is large, its calibration row is large, or both), so between 20 and 40 features. On Dataset C that is 35 (5 chosen by both), i.e. 3 500 weights. The score is the relative error of $W_{:,S}X_{\text{cal}}[S,:]$, not of the full product.

What this changes. A large calibration row can enter the score even when its weight column is not near the top of s_W, the case Version 1 misses. Because selection is by a whole column or a whole row, one large entry of X_{cal} no longer forces every weight of its column to be quantised: the column is included only when its row sum $s_X(i)$ is in the top 20 %.

4.5 Result on Dataset C

Sampled error: the partial product at baseline exponents. Full baseline: the same exponents on the complete product. Exhaustive search: the complete product after the exact exponent search. Rank 1 is best; ties share a rank.

	Sampled error (%)			Full baseline	Exhaustive search
Format	Version 1	Version 2	Version 3	error (%)	error (%)
E2M1	**12.484 (1)**	**12.248 (1)**	**12.552 (1)**	**12.552** (1)	**12.031** (1)
MXINT4	13.898 (2)	14.166 (2)	14.614 (2)	15.808 (2)	13.452 (2)
BFP4	13.898 (2)	14.166 (2)	14.614 (2)	15.808 (2)	13.452 (2)
E1M2	13.911 (4)	14.183 (4)	14.615 (4)	15.817 (4)	14.004 (5)
MXUINT4	15.024 (5)	15.329 (5)	15.744 (5)	17.126 (5)	13.452 (2)

Table 6: Dataset C. Sampled error of each version against the full baseline and the exhaustive search; rank in brackets.

Same winner. E2M1 is rank 1 for all three versions, for the full baseline and for the exhaustive search, so each sampled selector picks the exhaustive-search winner.

Close to the baseline, and cheaper. Version 3's sampled E2M1 error, 12.552 %, is within 0.001 percentage points of the full baseline's 12.552 % although it quantises only 3 500 of 10 000 weights; the exponent search then lowers it to 12.031 %.

Where the ranking differs. After the search MXINT4, BFP4 and MXUINT4 tie at 13.452 % and E1M2 is last at 14.004 %. No selector reproduces that tie: all place MXUINT4 last, because they rank at baseline exponents, where the MXUINT4 rule is the most conservative.

Conclusion. Column-wise K misses columns whose calibration row carries the energy; entry-wise masks avoid the full multiply but pull in a whole weight column whenever the matching calibration row is selected. The union of top weight columns and top calibration rows uses both sides of the product at feature level, and on Dataset C selects E2M1, the exhaustive-search winner.

5 Generic Algorithm 3: the Histogram Selector (GHS-2)

Brute force rounds all 10 000 weights in every format, and the exhaustive search repeats that for many exponent choices. Algorithm 3 first asks a cheaper question: *if we only knew how the weights are spread out, which format would lose least?* It answers from a **histogram** of the weights, so that no weight is rounded, and spends real rounding only where the histogram is not sure (Figure 1). The selector never calls the exhaustive search: the notebook checks this in code and uses the search only afterwards, to grade the answer. Formats and rounding are those of Section 2 (the notebook checks its fast quantisers against the scalar brute-force code on 4 500 blocks); non-finite inputs raise an error, and an all-zero W or X_{cal} ties every format at zero error (Stage 0).

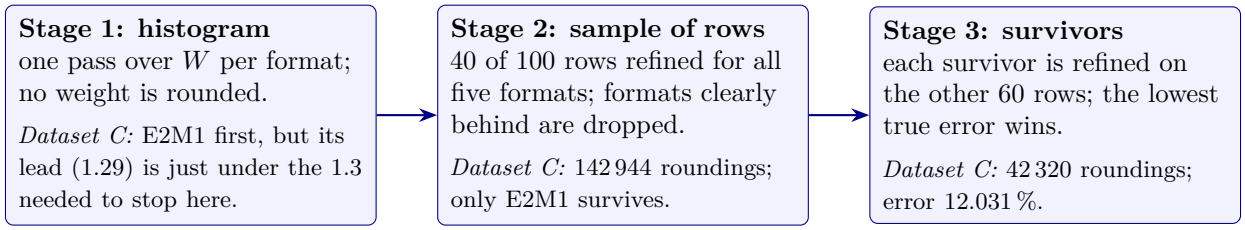


Figure 1: The stages of Algorithm 3 with the Dataset C numbers (Stage 0, the safety check, is not drawn).

5.1 How it works

Stage 1: histogram (no weight is rounded).

- *Make blocks comparable.* Each weight is divided by its block's baseline scale s, so its size t lies between 0 and 8.
- *Count, weighted by the inputs.* A weight W_{ij} adds the mass $a_j s^2$ to the bin of its size (a_j is Algorithm 1's input energy, so the mass is how much output error one unit of squared rounding error on that weight causes). Bins are 1/128 wide and line up with the rounding cells; weights are also counted by sign (integer formats) and by 8 block classes (where the block maximum sits in its octave), so each class can choose its own exponent.
- *Read the error from the bins.* Per bin, the total mass m_0, the sum of mass$\times t$ (m_1) and of mass$\times t^2$ (m_2) give the squared error of rounding the whole bin to a level q as $m_2 - 2q\,m_1 + q^2 m_0$. Adding this over the bins for each of 8 exponent offsets (-4 to 3) gives every class its best exponent, the format's score S_f and a starting exponent for every block, without touching a weight.
- *Stop or go on.* The margin is the second-best score over the best. A margin of at least 1.3 with weakly correlated inputs (largest eigenvalue under 10 % of the sum for $X_{\text{cal}}X_{\text{cal}}^\top$) accepts the leader at once; on Dataset C they are 1.29 and 2.5 %, so the algorithm goes on.

Stage 2: refine a sample of rows on the true error. Rows of W are independent: the error of row r is $\|(w_r - \hat{w}_r)F\|^2$ with $FF^\top = X_{\text{cal}}X_{\text{cal}}^\top$, so X_{cal} is never multiplied again. The sample is the heaviest rows by $\sum_j W_{ij}^2 a_j$ (half of the total, at most a quarter of the rows: 25 here, where 42 would be needed) plus 15 evenly spaced others: 40 rows. Each format starts from the histogram's exponents; one block's exponent, then any two blocks' of a row, moves by up to ± 3, and a move is kept only if the row's true error falls (two sweeps here). Sampled errors are scaled up to all rows; a format is dropped if it trails the sample leader by more than three standard errors (a paired z-test; a close competitor could still be dropped by mistake), but the histogram's leader always stays.

Stage 3: finish the survivors. Each survivor is refined on the remaining 60 rows; the lowest true error wins and its refined matrix is returned. Like the exhaustive search, GHS-2 thus returns

tuned block exponents, not the baseline-exponent matrix of Algorithms 1, 2 and 4.

5.2 Result against brute force

Errors are $100\,\|(W - W_{\mathrm{q}})X_{\mathrm{cal}}\|_F / \|WX_{\mathrm{cal}}\|_F$ (references: Section 3); tied formats share a rank. Stage 1 is shown as $100\sqrt{S_f}/\|WX_{\mathrm{cal}}\|_F$, Stage 2 as the scaled-up sample error.

Format	Stage 1 estimate (%)	Stage 2 sample (%)	Stage 3 error (%)	Full baseline error (%)	Exhaustive search error (%)
E2M1	**12.129** (1)	**12.201**	**12.031**	**12.552** (1)	**12.031** (1)
MXINT4	13.790 (2)	13.481	–	15.808 (2)	13.452 (2)
BFP4	13.790 (2)	13.481	–	15.808 (2)	13.452 (2)
E1M2	14.254 (4)	14.000	–	15.817 (4)	14.004 (5)
MXUINT4	15.272 (5)	13.481	–	17.126 (5)	13.452 (2)

Table 7: Dataset C; rank in brackets, ties share a rank. Raw Stage 1 scores: 372.5, 481.4, 514.4, 590.5 (E2M1, MXINT4 and BFP4, E1M2, MXUINT4).

Same winner, same error. The histogram ranks E2M1 first without a decisive lead (1.29) and orders the others only partly right (MXUINT4 last, against a tie for second). The Stage 2 sample puts E2M1 (12.201 %) at least 4.5 standard errors ahead of the rest (13.481–14.000 %), so only E2M1 reaches Stage 3, where its error 12.031 % equals the exhaustive search's to six decimals and is 4.2 % below the full baseline's 12.552 %.

Where the work goes. Stage 1 bins 50 000 weights but rounds none; Stages 2 and 3 round 142 944 and 42 320, 185 264 in all: 52.7 % of the notebook's reference run (351 800), or 52.9 % of the 350 400 of Table 5. It saves rounding, not multiplication: its error evaluations take 35 270 400 multiply-accumulates in the notebook's count, 2.8 times the 12 800 000 of five plain products.

5.3 Other inputs and limits

We re-ran the selector on four variants of Dataset C and graded it with the notebook's exhaustive search. With the 100 largest weights multiplied by 8, half of the weights zeroed, or row scales from 0.25 to 4, GHS-2 picks the exhaustive winner with an error within 0.001 percentage points of it, rounding 42–84 % as many weights as the exhaustive search; with outliers the histogram margin is only 1.01, so four formats reach Stage 3 and the saving shrinks. With correlated inputs (our construction: the 100 channels share one random signal carrying 90 % or 99 % of the variance) it still picks E2M1, but its matrix is 2.7 % and 16.0 % worse than the exhaustive search's: the histogram sees only each channel's input energy, not the correlation between channels, and local moves find only part of the cancellation across blocks that the exhaustive search exploits. So GHS-2 is a histogram-seeded local search, not an analytical rule, it can be beaten on correlated inputs, and it uses calibration data only.

6 Generic Algorithm 4: the Calibration-Column Sampling Selector

Algorithms 1 and 2 decide which part of W to look at: blocks, or input features. Algorithm 4 decides which *examples* to look at. The calibration matrix X_{cal} (100×256) holds 256 example inputs, one per column, and the output error of a format is a sum over those columns,

$$\|(W - W_{\text{q}})X_{\text{cal}}\|_F^2 = \sum_{k=1}^{256} \left\|(W - W_{\text{q}})\, X_{\text{cal}}[:, k]\right\|^2.$$

If a few columns represent the rest, the formats can be ranked from those columns alone. The selector keeps *all* of W and all five formats, but multiplies by 32 calibration columns instead of 256: the single column with the largest energy, plus 31 columns drawn at random. The steps below follow the notebook and its README.

6.1 How it works

Step 1: pick 32 calibration columns. The energy of column k is $e_k = \sum_i X_{ik}^2$. The column k^* with the largest energy is always kept (column 114, counting from 0, on Dataset C, with 1.49 times the mean energy): it can contribute the largest share of the error, and a random draw might miss it. Then 31 further columns are drawn without repetition from the other 255, with a fixed seed (271828), so the same columns are chosen on every run. With 32 columns or fewer, all of them are used. Only calibration data enter the selection; the held-out inputs never do.

Step 2: quantise every weight in every format. All 400 row-wise blocks of 32 weights are rounded in the five formats at their baseline exponent (Section 2), which is 50 000 values in total, so no weight is skipped. This gives the rounding error $\Delta_f = W_{\text{q}}^{(f)} - W$ of every format f.

Step 3: estimate the error. For each format, compare the outputs from the original and rounded weights using the 32 selected columns of X_{cal}. Square the differences and add them separately for the one largest-energy column and the 31 random columns. Call these totals E_{one} and E_{random}. The random columns represent the other 255 columns, so multiply their total by 255/31. The estimated RMSE is

$$s_f = \sqrt{\frac{E_{\text{one}} + (255/31)E_{\text{random}}}{100 \times 256}}.$$

The denominator is the number of values in the full output. A smaller score is better.

Step 4: choose the format. Choose the format with the smallest estimated RMSE. If several formats have nearly equal lowest scores, check those formats using all 256 columns and choose the one with the lowest full RMSE. Scores are considered tied when $|s - s'| \le 10^{-15} + 10^{-6} \max(|s|, |s'|)$. If a tie remains, use the fixed format order. On Dataset C, no full-column tie check is needed. Return the chosen format's rounded weight matrix using its baseline scales; no scale search is performed.

On Dataset C there is no tie, so this step is not used. The result is the winner's baseline-quantised matrix; no exponent search is run.

6.2 Result against brute force

The full baseline and the exhaustive search are the two brute-force references of Section 3. Errors are $100\,\|(W - W_{\mathrm{q}})X_{\mathrm{cal}}\|_F / \|WX_{\mathrm{cal}}\|_F$ and tied formats share a rank; the selector's score is the 32-column RMSE of Step 3 as the notebook prints it.

	Selector (32 columns)		Full baseline		Exhaustive search	
Format	score	rank	error (%)	rank	error (%)	rank
E2M1	**0.1247**	1	**12.552**	1	**12.031**	1
MXINT4	0.1631	2	15.808	2	13.452	2
BFP4	0.1631	2	15.808	2	13.452	2
E1M2	0.1633	4	15.817	4	14.004	5
MXUINT4	0.1739	5	17.126	5	13.452	2

Table 8: Dataset C. The selector's score is the estimated output RMSE (not a percentage); the other two columns are full output errors.

Same winner, with a clear margin. E2M1 is rank 1 for the sampled score, the full baseline and the exhaustive search. Its lead over the runner-up is 30.8 % in the sampled score, 25.9 % at baseline exponents and 11.8 % after the search. The sampled ranking equals the full baseline's ($\tau_b = 1.00$) but only partly the exhaustive one ($\tau_b = 0.63$): MXUINT4 is last at baseline exponents but ties MXINT4 and BFP4 once the exponents are searched. The delivered matrix is the baseline E2M1 matrix with error 12.552 %; the exhaustive search lowers it to 12.031 %, a gain this selector does not attempt.

The estimate is close. Converted to the percentage unit of the error columns, the 32-column estimate for E2M1 is 12.542 % against the exact 12.552 % (RMSE 0.1247 against 0.1248). The other four formats are over-estimated by 2.1–3.8 %, which keeps their order. The pick does not hinge on a lucky draw: over the seeds 0 to 999 (listed in advance, not chosen by their result) E2M1 wins 1 000 times, its estimate stays within 0.93 to 1.05 times the exact value, and the runner-up is never closer than 18.6 % in score.

Less work, but not less rounding. Using 32 of 256 columns cuts the sampled product to 12.5 % of a full pass, yet all 50 000 values are still rounded. In the operation count used for Algorithm 1 the selector needs 3 616 965 operations, 13.9 % of the full baseline's 26 078 012 (88.0 % sampled product, 8.3 % rounding, the rest column energies and the differences Δ_f). Algorithm 1 spends 39.0 % but rounds only 50.1 % of the values: Algorithm 1 saves on the rounding, Algorithm 4 on the product.

When it can fail. The estimate is good here because the columns are fairly even: the largest column error of E2M1 is only 1.65 times the mean, so 32 columns cover the spread. Heavier-tailed calibration data could hide large columns that the draw misses. A near-tie also costs a full product per tied finalist (19.7 % of a baseline pass each), which erodes the saving. The scores use calibration data only (no held-out test) and one dataset.

References

[1] Open Compute Project, *OCP Microscaling Formats (MX) Specification v1.0*, 2023. https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf

[2] B. D. Rouhani et al., *Microscaling Data Formats for Deep Learning*, arXiv:2310.10537, 2023.

[3] R. Wang et al., *Optimizing Large Language Model Training Using FP4 Quantization*, ICML (PMLR 267), 2025, Table 4.

[4] U. Köster et al., *Flexpoint: An Adaptive Numerical Format for Efficient Training of Deep Neural Networks*, NeurIPS 2017, arXiv:1711.02213.